Université Protestante au Congo

Faculté de Sciences informatiques

Travaux Pratiques de Programmation web en PHP

Système de Facturation avec Lecture de Codes-Barres

Programmation Procédurale – Gestion de Fichiers – Contrôle d'Accès

Année :	2025 – 2026
Mode de travail :	Groupes de 3 étudiants maximum

Table des matières

1 Contexte et Objectifs **3**
1.1 Mise en situation . . . 3
1.2 Objectifs pédagogiques . . . 3
1.3 Contrainte fondamentale . . . 3

2 Structure du Projet **3**
2.1 Arborescence imposée . . . 3
2.2 Description sommaire des répertoires . . . 5

3 Partie 1 – Enregistrement des Produits **5**
3.1 Description fonctionnelle . . . 5
3.2 Comportement attendu . . . 5
3.3 Structure d'un produit . . . 5
3.4 Contraintes techniques . . . 6

4 Partie 2 – Facturation **6**
4.1 Description fonctionnelle . . . 6
4.2 Comportement attendu . . . 6
4.3 Format d'affichage de la facture . . . 6
4.4 Structure d'une facture . . . 7
4.5 Contraintes techniques . . . 7

5 Partie 3 – Gestion des Comptes Utilisateurs **7**
5.1 Description fonctionnelle . . . 7
5.2 Rôles et permissions . . . 7
5.3 Structure d'un utilisateur . . . 8
5.4 Contraintes techniques . . . 8

6 Partie 4 – Rapport Technique **8**
6.1 Contenu obligatoire . . . 8
6.2 Exigences de rédaction . . . 8

7 Modalités de Rendu **9**
7.1 Fichiers à remettre . . . 9
7.2 Critères d'évaluation . . . 9

Avertissement

Ce travail pratique est en groupe des 3 personnes par groupe. Tout plagiat, toute copie partielle ou totale du code d'un autre groupe entraînera la note de zéro pour les deux parties concernées. Le code soumis doit être entièrement rédigé et compris par les étudiants qui le remettent.

1 Contexte et Objectifs

1.1 Mise en situation

Un petit super marché souhaite moderniser son processus de vente. Actuellement, les caissiers saisissent manuellement les articles, ce qui génère des erreurs et ralentit les transactions. Le gérant souhaite mettre en place un système de caisse informatisé, accessible depuis un navigateur web, capable de lire des codes-barres via la caméra d'un téléphone ou d'un ordinateur, et de produire des factures détaillées.

Vous êtes mandatés pour développer ce système en PHP procédural, sans recours à aucun système de gestion de base de données.

1.2 Objectifs pédagogiques

À l'issue de ce projet, l'étudiant sera capable de :

- Concevoir et structurer une application web PHP selon le paradigme procédural
- Utiliser les fichiers comme mécanisme de persistance des données
- Implémenter un système d'authentification et de contrôle d'accès basé sur les rôles
- Intégrer une bibliothèque de lecture de codes-barres dans une interface web
- Appliquer les bonnes pratiques de validation et d'assainissement des données en PHP
- Rédiger un rapport technique structuré en LATEX

1.3 Contrainte fondamentale

Toute la persistance des données doit être assurée **exclusivement par des fichiers** au format que vous allez définir. L'utilisation d'un système de gestion de base de données (MySQL, PostgreSQL, SQLite, MongoDB, etc.) est **strictement interdite** et entraînera la note de zéro pour la partie concernée.

2 Structure du Projet

2.1 Arborescence imposée

Il est suggéré d'organiser votre projet selon l'arborescence ci-dessous, présentée comme un exemple de structuration. Vous pouvez l'adapter en fonction de vos besoins, à condition de justifier clairement toute modification dans le rapport.

```
facturation/
├── index.php
├── config/
│   └── config.php
├── auth/
│   ├── login.php
│   ├── logout.php
│   └── session.php
├── modules/
│   ├── produits/
│   │   ├── enregistrer.php
│   │   ├── lire.php
│   │   └── liste.php
│   ├── facturation/
│   │   ├── nouvelle-facture.php
│   │   ├── calcul.php
│   │   └── afficher-facture.php
│   └── admin/
│       ├── gestion-comptes.php
│       ├── ajouter-compte.php
│       └── supprimer-compte.php
├── data/
│   ├── produits.json
│   ├── factures.json
│   └── utilisateurs.json
├── includes/
│   ├── header.php
│   ├── footer.php
│   ├── fonctions-produits.php
│   ├── fonctions-factures.php
│   └── fonctions-auth.php
├── assets/
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── scanner.js
└── rapports/
    ├── rapport-journalier.php
    └── rapport-mensuel.php
```

2.2 Description sommaire des répertoires

Répertoire	Rôle
`config/`	Paramètres globaux (taux TVA, chemins des fichiers, etc.)
`auth/`	Gestion de l'authentification et des sessions
`modules/`	Modules fonctionnels (produits, facturation, administration)
`data/`	Fichiers de persistance des données
`includes/`	Fonctions PHP réutilisables incluses dans les pages
`assets/`	Ressources statiques (CSS, JavaScript)
`rapports/`	Génération des rapports journaliers et mensuels

Dans votre rapport, vous devez décrire le rôle précis de **chaque fichier** de cette arborescence.

3 Partie 1 – Enregistrement des Produits

3.1 Description fonctionnelle

Cette partie concerne l'alimentation du catalogue produits. Lorsqu'un produit est présenté pour la première fois au système, son code-barres est inconnu. Le module d'enregistrement permet alors de l'associer à ses informations commerciales.

3.2 Comportement attendu

1. L'utilisateur active la caméra via l'interface web.
2. La bibliothèque JavaScript lit le code-barres et transmet sa valeur au serveur PHP.
3. Le serveur PHP consulte `data/produits.json` pour vérifier si le code-barres est déjà référencé.
4. Si le code-barres est **inconnu**, un formulaire s'affiche avec les champs :
   — Nom du produit
   — Prix unitaire hors taxe (en CDF)
   — Date d'expiration (format `MM-JJ-AAAA`)
   — Quantité initiale en stock
5. Après validation, les données sont écrites dans `data/produits.json`.
6. Si le code-barres est **déjà connu**, les informations existantes s'affichent.

3.3 Structure d'un produit

```
{
  "code_barre": "3017620422003",
  "nom": "Vain amour 1L",
  "prix_unitaire_ht": 1200,
  "date_expiration": "2026-12-31",
  "quantite_stock": 50,
```

```
  "date_enregistrement": "2026-04-17"
}
```

3.4 Contraintes techniques

- La lecture du code-barres est réalisée via `ZXing` ou `QuaggaJS`.
- Toutes les données saisies doivent être validées **côté serveur** avant tout enregistrement.
- Vérifications minimales : champs non vides, prix et quantité numériques et positifs, format de date valide.
- En cas de saisie invalide, un message d'erreur s'affiche sans effacer les données déjà saisies.
- Accès réservé aux rôles **Manager** et **Super Administrateur**.

4 Partie 2 – Facturation

4.1 Description fonctionnelle

Ce module est le cœur opérationnel du système. Il permet au caissier de créer une facture en scannant successivement les codes-barres des articles vendus.

4.2 Comportement attendu

1. Le caissier scanne le code-barres d'un produit.
2. Le serveur PHP recherche le produit dans `data/produits.json`.
3. Si trouvé, le nom et le prix unitaire HT s'affichent automatiquement.
4. Le caissier saisit la quantité vendue.
5. Le système ajoute la ligne à la facture et recalcule le récapitulatif.
6. L'opération se répète pour chaque article supplémentaire.
7. À la validation, le système affiche :
   - Désignation, prix unitaire HT, quantité, sous-total HT par article
   - Total HT, montant TVA, total TTC et net à payer
8. La facture est horodatée et sauvegardée dans `data/factures.json`.

4.3 Format d'affichage de la facture

Désignation	**Prix unit. HT**	**Qté**	**Sous-total HT**
Huile de palme 1L	1 200 CDF	2	2 400 CDF
Savon de Marseille	450 CDF	5	2 250 CDF
Total HT			4 650 CDF
TVA (18%)			837 CDF
Net à payer			**5 487 CDF**

4.4 Structure d'une facture

```
{
  "id_facture": "FAC-20260417-001",
  "date": "2026-04-17",
  "heure": "10:35:22",
  "caissier": "dan.mbo",
  "articles": [
    {
      "code_barre": "3017620422003",
      "nom": "Vain amour",
      "prix_unitaire_ht": 1200,
      "quantite": 10,
      "sous_total_ht": 2400
    }
  ],
  "total_ht": 2400,
  "tva": 432,
  "total_ttc": 2832
}
```

4.5 Contraintes techniques

- Produit inconnu : message invitant le Manager à l'enregistrer préalablement.
- Quantité supérieure au stock : alerte affichée et transaction bloquée.
- Le stock est décrémenté dans `data/produits.json` après validation.
- Chaque facture reçoit un identifiant unique généré automatiquement.

5 Partie 3 – Gestion des Comptes Utilisateurs

5.1 Description fonctionnelle

Le système implémente un contrôle d'accès basé sur les rôles (RBAC) avec trois niveaux hiérarchiques. Chaque page de l'application vérifie le rôle de l'utilisateur connecté avant d'en autoriser l'accès.

5.2 Rôles et permissions

Rôle	Permissions
Caissier	Lecture de codes-barres, création et consultation de factures
Manager	Toutes les permissions du Caissier, plus : enregistrement de produits, modification du stock, consultation des rapports
Super Administrateur	Toutes les permissions du Manager, plus : création et suppression de comptes Caissiers et Managers

5.3 Structure d'un utilisateur

```
{
  "identifiant": "ezechiel.itewe",
  "mot_de_passe": "$Freedom@23...",
  "role": "caissier",
  "nom_complet": "Ezechiel Itewe",
  "date_creation": "2026-04-17",
  "actif": true
}
```

Le champ `mot_de_passe` contient le hachage produit par `password_hash()`.

5.4 Contraintes techniques

- Hachage avec `password_hash()` et vérification avec `password_verify()`.
- Sessions PHP (`session_start()`) pour maintenir l'état de connexion.
- Chaque fichier PHP inclut `auth/session.php` et vérifie le rôle requis.
- Utilisateur non connecté : redirection automatique vers `auth/login.php`.
- Accès non autorisé : message d'erreur et redirection vers la page d'accueil du rôle.
- Le compte Super Administrateur initial est créé manuellement dans `data/utilisateurs.json`.

6 Partie 4 – Rapport Technique

6.1 Contenu obligatoire

1. **Page de garde** : noms des étudiants, niveau, matière, année académique.
2. **Introduction** : présentation du projet, contexte, objectifs.
3. **Arborescence commentée** : description précise du rôle de chaque fichier.
4. **Structures de données** : justification des formats utilisés.
5. **Description des modules** : logique du code et justification des fonctions PHP employées.
6. **Diagramme de flux** : traitement complet d'une transaction de vente.
7. **Difficultés rencontrées et solutions apportées**.
8. **Conclusion et pistes d'amélioration**.

6.2 Exigences de rédaction

- Minimum **15 pages** hors page de garde et table des matières.
- Le rapport doit être en Latex.
- Il est recommandé d'éviter les captures d'écran dans le rapport.
- Le code source est idéalement hébergé sur une plateforme dédiée telle que **GitHub** et référencé dans le rapport, plutôt que d'y être directement intégré.

7 Modalités de Rendu

7.1 Fichiers à remettre

Contenu obligatoire :

- Code source complet sur dépôt GITHUB
- Le rapport final est à transmettre par courrier électronique à l'adresse suivante : `programmationwebphp@gmail.com`. au format PDF
- Fichier `README.txt` avec les instructions de déploiement local

7.2 Critères d'évaluation

Critère	**Points**
Lecture et enregistrement de codes-barres	4
Module de facturation (calculs, affichage, mise à jour stock)	5
Gestion des comptes et contrôle d'accès	4
Persistance des données par fichiers	3
Qualité et clarté du code procédural PHP	2
Rapport en Latex (structure, explications, arborescence)	2
Total	**20**

Date limite de remise : Jeudi 24 Avril 2026 à 23h59

Défense vendredi 25 Avril 2026

Parce que la réussite se forge par l'effort et la persévérance, nous vous souhaitons beaucoup de succès ainsi qu'un agréable moment de vibe coding. FREEDOM